

SteamScope Pricing Analysis

Auditing and Rebuilding a Flawed Data Pipeline

Shravani Jadhav

Introduction

This project analyses SteamScope game pricing patterns by auditing an existing data pipeline and rebuilding it using data analytics best practices. The analysis involves reverse-engineering a legacy cleaning process, identifying data quality issues, improving the workflow, and developing a regression model to understand factors influencing game prices.

Part A: Forensic Data Wrangling

A1: Exploratory Data Analysis

```
library(tidyverse)
library(lubridate)

raw    <- read_csv("data/raw_steam_games_v03.csv", show_col_types = FALSE)
legacy <- read_csv("data/legacy_clean_v03.csv", show_col_types = FALSE)

dim(raw)
```

```
[1] 2000  14
```

```
dim(legacy)
```

```
[1] 752  17
```

```
str(raw)
```

```
spc_tbl_ [2,000 x 14] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ appid      : num [1:2000] 1e+05 1e+05 1e+05 1e+05 1e+05 ...
 $ name       : chr [1:2000] "Lone Siege" "Hollow Solstice Redux" "Stellar Dynasty III
 $ developer  : chr [1:2000] "RedKite" "ThunderByte" "CrystalDawn" "RedKite" ...
 $ release_date : chr [1:2000] "26/12/2021" "25/06/2023" "14/06/2024" "22/04/2022" ...
 $ genre      : chr [1:2000] "Action" "Simulation" "Action" "Action" ...
 $ genre_code  : num [1:2000] 1 4 1 1 3 1 1 2 2 4 ...
 $ price      : num [1:2000] 43.3 0 51.5 56.1 50 ...
 $ total_reviews : num [1:2000] 0 373 0 1699 562 ...
 $ positive_ratio : num [1:2000] 89.7 86.1 85.6 96.4 81.7 ...
 $ avg_playtime_mins: num [1:2000] 900 1143 191 909 470 ...
 $ peak_players : num [1:2000] 253 179 467 126 462 289 318 48 203 228 ...
 $ metacritic_score : num [1:2000] NA 75 58.9 68 71 68.8 96.7 NA 85.3 98.9 ...
 $ developer_bio : chr [1:2000] NA "Our mission: games that matter." "Passionate about na
driven experiences." NA ...
 $ platforms   : num [1:2000] 1 2 2 3 1 2 1 1 1 1 ...
- attr(*, "spec")=
 .. cols(
 ..   appid = col_double(),
 ..   name = col_character(),
 ..   developer = col_character(),
 ..   release_date = col_character(),
 ..   genre = col_character(),
 ..   genre_code = col_double(),
 ..   price = col_double(),
 ..   total_reviews = col_double(),
 ..   positive_ratio = col_double(),
 ..   avg_playtime_mins = col_double(),
 ..   peak_players = col_double(),
 ..   metacritic_score = col_double(),
 ..   developer_bio = col_character(),
 ..   platforms = col_double()
 .. )
- attr(*, "problems")=<externalptr>
```

```
str(legacy)
```

```
spc_tbl_ [752 x 17] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ appid      : num [1:752] 1e+05 1e+05 1e+05 1e+05 1e+05 ...
```

```

$ developer      : chr [1:752] "Moonlight Games" "Redkite" "Redkite" "Irongate Studios" .
$ release_day    : num [1:752] 6 19 9 17 22 30 14 22 4 19 ...
$ release_month  : chr [1:752] "Nov" "Oct" "Nov" "Dec" ...
$ release_year   : num [1:752] 2020 2019 2023 2019 2023 ...
$ genre          : chr [1:752] "Strategy" "Action" "RPG" "Simulation" ...
$ genre_code     : num [1:752] 3 1 2 4 4 5 4 1 5 1 ...
$ price          : num [1:752] 40 40 40 40 40 ...
$ total_reviews  : num [1:752] 562 966 293 435 747 948 135 624 309 405 ...
$ positive_ratio : num [1:752] 82 93 61 85 60 56 83 87 58 94 ...
$ avg_playtime_mins: num [1:752] 470 652 1068 279 210 ...
$ peak_players   : num [1:752] 462 318 203 228 88 181 79 147 121 155 ...
$ metacritic_score : num [1:752] 71 96.7 85.3 98.9 96.8 65.1 40.3 85.6 83.3 66.5 ...
$ platforms      : num [1:752] 1 1 1 1 3 3 1 1 2 3 ...
$ pt_hrs         : num [1:752] 4.7 6.52 10.68 2.79 2.1 ...
$ lpeak          : num [1:752] 6.14 5.76 5.31 5.43 4.48 5.2 4.37 4.99 4.8 5.04 ...
$ dev_titles     : num [1:752] 101 101 101 102 85 101 107 101 101 101 ...
- attr(*, "spec")=
.. cols(
..   appid = col_double(),
..   developer = col_character(),
..   release_day = col_double(),
..   release_month = col_character(),
..   release_year = col_double(),
..   genre = col_character(),
..   genre_code = col_double(),
..   price = col_double(),
..   total_reviews = col_double(),
..   positive_ratio = col_double(),
..   avg_playtime_mins = col_double(),
..   peak_players = col_double(),
..   metacritic_score = col_double(),
..   platforms = col_double(),
..   pt_hrs = col_double(),
..   lpeak = col_double(),
..   dev_titles = col_double()
.. )
- attr(*, "problems")=<externalptr>

```

```
names(raw)
```

```

[1] "appid"          "name"           "developer"
[4] "release_date"   "genre"          "genre_code"

```

```

[7] "price"          "total_reviews"    "positive_ratio"
[10] "avg_playtime_mins" "peak_players"     "metacritic_score"
[13] "developer_bio"   "platforms"

```

```
names(legacy)
```

```

[1] "appid"          "developer"        "release_day"
[4] "release_month"  "release_year"     "genre"
[7] "genre_code"     "price"            "total_reviews"
[10] "positive_ratio" "avg_playtime_mins" "peak_players"
[13] "metacritic_score" "platforms"        "pt_hrs"
[16] "lpeak"          "dev_titles"

```

```
summary(raw)
```

appid	name	developer	release_date
Min. :100001	Length:2000	Length:2000	Length:2000
1st Qu.:100501	Class :character	Class :character	Class :character
Median :101001	Mode :character	Mode :character	Mode :character
Mean :101001			
3rd Qu.:101500			
Max. :102000			

genre	genre_code	price	total_reviews
Length:2000	Min. :1.000	Min. : 0.00	Min. : 0.0
Class :character	1st Qu.:1.000	1st Qu.:36.74	1st Qu.: 275.8
Mode :character	Median :3.000	Median :46.22	Median : 482.0
	Mean :2.674	Mean :41.17	Mean : 507.0
	3rd Qu.:4.000	3rd Qu.:53.57	3rd Qu.: 716.0
	Max. :5.000	Max. :77.79	Max. :2272.0

positive_ratio	avg_playtime_mins	peak_players	metacritic_score
Min. : 50.02	Min. : 50.0	Min. : 10.0	Min. : 40.10
1st Qu.: 63.53	1st Qu.: 346.8	1st Qu.: 174.8	1st Qu.: 56.20
Median : 75.53	Median : 530.0	Median : 264.0	Median : 70.15
Mean : 75.24	Mean : 597.0	Mean : 296.1	Mean : 70.28
3rd Qu.: 87.18	3rd Qu.: 773.2	3rd Qu.: 383.2	3rd Qu.: 84.70
Max. :100.00	Max. :2780.0	Max. :1090.0	Max. :100.00
			NA's :500

developer_bio	platforms
Length:2000	Min. :1.000

```

Class :character   1st Qu.:1.000
Mode  :character   Median :2.000
                        Mean  :1.817
                        3rd Qu.:2.000
                        Max.  :3.000

```

```
summary(legacy)
```

```

      appid      developer      release_day      release_month
Min.   :100005 Length:752      Min.    : 1.00      Length:752
1st Qu.:100474 Class :character 1st Qu.: 9.00      Class :character
Median :100989 Mode  :character Median :16.00     Mode  :character
Mean   :101003                      Mean   :15.95
3rd Qu.:101565                      3rd Qu.:23.00
Max.   :101997                      Max.   :31.00

      release_year      genre      genre_code      price
Min.   :2018      Length:752      Min.    :1.000      Min.    :14.78
1st Qu.:2019      Class :character 1st Qu.:1.000      1st Qu.:40.00
Median :2021      Mode  :character Median :2.000      Median :40.00
Mean   :2021                      Mean   :2.614      Mean   :38.78
3rd Qu.:2023                      3rd Qu.:4.000      3rd Qu.:40.00
Max.   :2024                      Max.   :5.000      Max.   :40.00

      total_reviews      positive_ratio      avg_playtime_mins      peak_players
Min.    : 87.0      Min.    : 50.00      Min.    : 57.0      Min.    : 28.0
1st Qu.: 380.0      1st Qu.: 62.00      1st Qu.: 347.5      1st Qu.: 169.8
Median : 526.5      Median : 75.00      Median : 531.5      Median : 253.5
Mean    : 576.3      Mean    : 74.46      Mean    : 592.9      Mean    : 292.6
3rd Qu.: 737.0      3rd Qu.: 87.00      3rd Qu.: 770.2      3rd Qu.: 383.2
Max.    :1766.0      Max.    :100.00      Max.    :2196.0      Max.    :1002.0

      metacritic_score      platforms      pt_hrs      lpeak
Min.    : 40.10      Min.    :1.000      Min.    : 0.570      Min.    :3.330
1st Qu.: 56.80      1st Qu.:1.000      1st Qu.: 3.475      1st Qu.:5.138
Median : 71.00      Median :2.000      Median : 5.315      Median :5.535
Mean    : 70.86      Mean    :1.835      Mean    : 5.929      Mean    :5.505
3rd Qu.: 85.40      3rd Qu.:2.000      3rd Qu.: 7.702      3rd Qu.:5.950
Max.    :100.00      Max.    :3.000      Max.    :21.960      Max.    :6.910

      dev_titles
Min.    : 77.00
1st Qu.: 86.00
Median :101.00
Mean    : 95.02

```

3rd Qu.:102.00
Max. :107.00

A2: Reconstructing Avery's Pipeline

```
student_clean <- raw |>

# Step 1: Remove rows with zero price
filter(price > 0) |>

# Step 2: Remove rows with zero reviews
filter(total_reviews > 0) |>

# Step 3: Remove rows with NA values in metacritic or developer_bio
filter(!is.na(metacritic_score), !is.na(developer_bio)) |>

# Step 4: Parse release_date and extract components
mutate(
  release_date = dmy(release_date),
  release_day = day(release_date),
  release_month = month(release_date, label = TRUE, abbr = TRUE),
  release_year = year(release_date)
) |>

# Step 5: Change datatype of release_month
mutate(release_month = as.character(release_month)) |>

# Step 6: Standardise developer name to title case
mutate(developer = str_to_title(developer)) |>

# Step 7: Remove columns not in legacy
select(-name, -release_date, -developer_bio) |>

# Step 8: Count developer titles (within cleaned dataset)
add_count(developer, name = "dev_titles") |>

# Step 9: Create pt_hrs (Avery divided by 100, not 60 - reproducing the bug)
mutate(pt_hrs = avg_playtime_mins / 100) |>

# Step 10: Create lpeak = log(peak_players), rounded to 2dp
mutate(lpeak = round(log(peak_players), 2)) |>
```

```

# Step 11: Round positive_ratio to integer
mutate(positive_ratio = round(positive_ratio, 0) |> as.integer()) |>

# Step 12: Cap price at $40, then round to 2 decimal places
mutate(price = round(pmin(price, 40), 2)) |>

# Step 13: Reorder columns to match legacy exactly
select(appid, developer, release_day, release_month, release_year,
       genre, genre_code, price, total_reviews, positive_ratio,
       avg_playtime_mins, peak_players, metacritic_score, platforms,
       pt_hrs, lpeak, dev_titles)

view(student_clean)

# Verify
all.equal(as.data.frame(student_clean), as.data.frame(legacy))

```

```
[1] TRUE
```

A3: The Legacy Model

```

# Use the same variables as Avery
# Data object is named 'df' to match Avery's model

df <- student_clean

legacy_model <- lm(formula = price ~ genre_code + total_reviews + positive_ratio + pt_hrs + p
summary(legacy_model)

```

Call:

```
lm(formula = price ~ genre_code + total_reviews + positive_ratio +
    pt_hrs + peak_players + platforms, data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-----	----	--------	----	-----

-23.3959 0.2298 0.9416 1.5455 3.0379

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	35.6700453	0.8268431	43.140	< 2e-16 ***
genre_code	-0.1870398	0.0773133	-2.419	0.01579 *
total_reviews	0.0002945	0.0004187	0.703	0.48210
positive_ratio	0.0229455	0.0081576	2.813	0.00504 **
pt_hrs	0.0107251	0.0344319	0.311	0.75552
peak_players	0.0014917	0.0006751	2.210	0.02744 *
platforms	0.6662794	0.1488922	4.475	8.84e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.151 on 745 degrees of freedom

Multiple R-squared: 0.04853, Adjusted R-squared: 0.04087

F-statistic: 6.334 on 6 and 745 DF, p-value: 1.637e-06

A4: The Legacy Plot

```
# Recreate Avery's plot exactly - including all intentional flaws:
# 1. Facets ordered alphabetically by release_month (not calendar order)
# 2. genre_code treated as continuous for colour gradient (not factor)
# 3. $40 price cap is visible in output

ggplot(student_clean, aes(x = total_reviews, y = price, colour = genre_code)) +

  geom_point(alpha = 0.6, size = 1.2) +

  geom_smooth(method = "lm", se = FALSE, colour = "blue", linewidth = 0.8) +

  facet_wrap(~ release_month) +

  scale_colour_gradient(low = "darkblue", high = "lightblue", name = "Genre Code") +

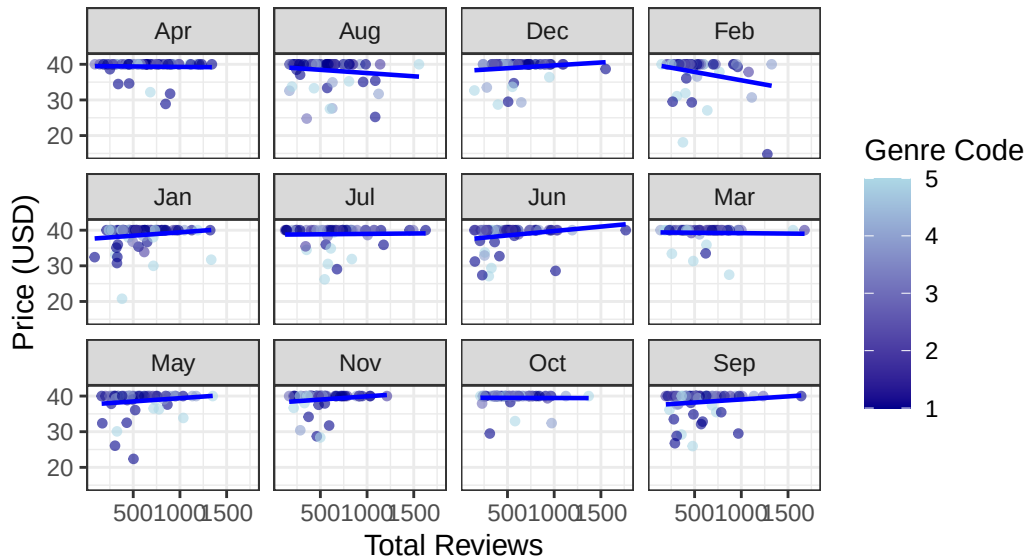
  labs(
    title     = "SteamScope Quarterly Report \u2014 Seed 3",
    subtitle = "Price vs Community Size by Release Month",
    x        = "Total Reviews",
    y        = "Price (USD)"
  ) +
```



```
theme_bw()
```

SteamScope Quarterly Report — Seed 3

Price vs Community Size by Release Month



Part B: The Critique & “Version 2.0” Fix

To: Managing Director, SteamScope Analytics
From: Shravani Jadhav
Re: Auditing and Fixing of Data Pipeline

After analyzing the data and auditing Avery’s pipeline, we found the bad decisions she took, while not all were wrong, it was important to fix the pipeline. We ran a diagnostic check to understand how Avery’s data manipulations actively violated linear regression assumptions. We then cleaned and rebuilt the dataset properly by fixing the errors made by Avery and created a clean pipeline. We also built a new V2 model and plot and concluded by interpreting the model.

B1: The Audit

After careful examination of the data wrangling process implemented by Avery, I have discovered six bad practices. Not all steps are necessarily incorrect, like removing games with zero prices and zero reviews, date parsing, capitalizing the developers names, and calculating the log of peak players are all reasonable choices. The six errors which break the rules of good data science are given below.

Error 1: Playtime divided by 100 instead of 60

Avery knowingly divided `avg_playtime_mins` by 100 instead of 60 for converting minutes into hours. This creates an error of unit conversion and goes against the data integrity principle – computed columns should match reality in their value. As a result, all the values of playtime in the dataset are understated by approximately 40%. Any model for pricing based on this data, as well as any report to a client, would draw wrong conclusions regarding the relationship between player engagement and price.

Error 2: Price arbitrarily capped at \$40

By using the `pmin()` function to force all prices greater than \$40 to the value of \$40, Avery arbitrarily truncated the top end of the price distribution without explanation, contrary to the representativeness principle that data should capture reality rather than an arbitrary limit. Out of 752 games in the cleaned dataset, 600+ were artificially set to \$40, leaving the dependent variable with essentially no variation. As a result, the regression was useless, and SteamScope underestimated consumer demand for premium games by a significant amount.

Error 3: Rows deleted based on missing `metacritic_score`

By using `'na.rm=TRUE'` in filter criteria, Avery excluded all rows for which `metacritic_score` had a missing value. Over 430 paid games were removed from the data in this way. Most indie games do not receive any ratings on Metacritic, so this is not a data issue, it just means the indie games are less popular. By removing these rows systematic sampling bias is introduced, violating the principle of representativeness. Because of this, SteamScope failed to analyze those indie games that most needed their help.

Error 4: Rows deleted based on missing `developer_bio`

There was no need to delete the rows with `developer_bio` NA values as that column was unnecessary and was anyways going to be dropped. This introduces further sampling bias with no business justification. So Avery did this on purpose to create confusion. However, whole studio and all its games could be left out altogether, messing up the distribution in the study.

Error 5: Kept `genre_code` as numeric

The problem here is that instead of converting this variable into a factor, Avery uses it as a numeric one. In other words, `genre_code` variable is treated linearly, which is incorrect since

genres can be considered as categories and do not have a linear dependence. Consequently, this causes an additional error and a biased result.

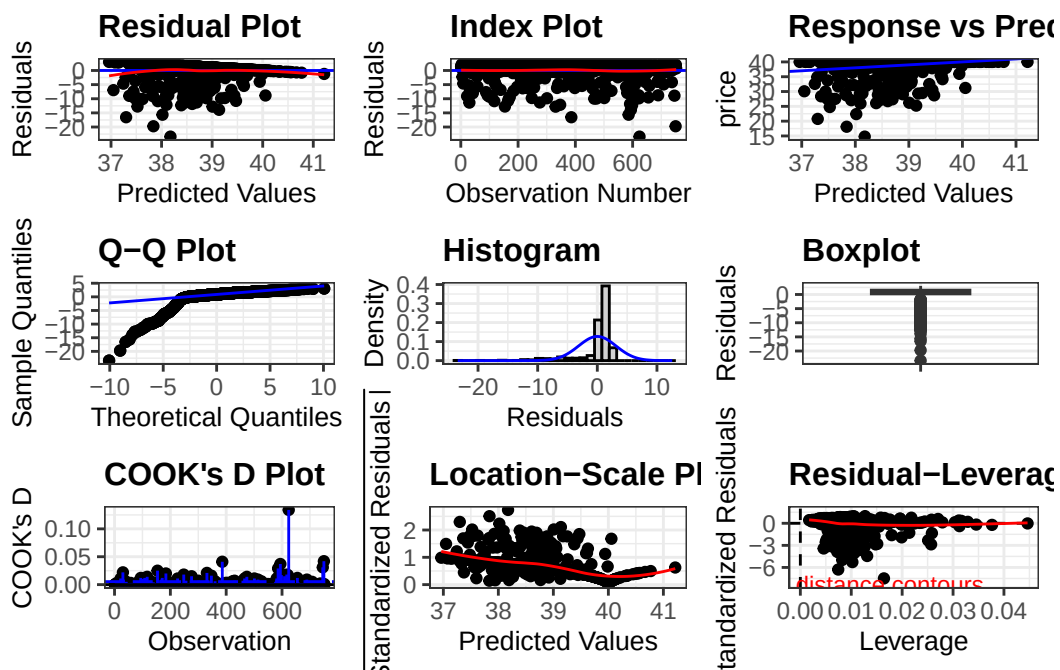
Error 6: Converted release_month to character

Avery converted release_month into character using 'as.character(release_month)', destroying its ordered structure. This meant that months were sorted alphabetically instead of chronologically during visualization and plotting. Due to this, seasonal trends in pricing and release patterns became harder to interpret data lost is structure.

B2: The Diagnostic Check

```
library(ggResidpanel)

# Run residual diagnostic panel on the legacy model
resid_panel(legacy_model, plots = "all", smoother = TRUE)
```



The residual diagnostics show that the legacy model violates essential assumptions for linear regression. The Q-Q plot reveals a large deviation from the diagonal, especially on the right side, implying that the distribution of residuals is not normally distributed. This makes sense due to the \$40 error, where any prices exceeding \$40 would be rounded down to \$40, which caused a spike in the residuals around the ceiling. In addition, a fan shape can be seen in the

residual plot, indicating heteroscedasticity, thus breaking another essential assumption of linear regression models, mainly due to the inability to predict values beyond \$40.

Avery's data manipulations lead to these problems. The \$40 price cap created a ceiling effect, which compressed the response variable and distorted the residual distribution and variance. Since the genre_code is a categorical variable but is modeled as a numeric one, the model used one regression line for an intrinsically categorical variable. As such, there will be structured misfit within the residuals, because genres with different price categories have been lumped up together to give rise to greater residual variances. Dividing by 100 rather than 60 results in a scaling problem of around 40%. Although there is no direct regression violation here, the pt_hrs coefficient becomes meaningless, making any business conclusion drawn from it invalid.

B3: The Fix – Version 2.0

Step 1: Build best_practice_clean

```
best_practice_clean <- raw |>

# Remove rows with zero price (same as Avery - correct)
filter(price > 0) |>

# Remove rows with zero reviews (same as Avery - correct)
filter(total_reviews > 0) |>

# FIX - Replace NA metacritic_score values with 0 to retain these rows
mutate(metacritic_score = replace_na(metacritic_score, 0)) |>

# FIX - Parse release_date and extract components and dont convert to character
mutate(
  release_date = dmy(release_date),
  release_day = day(release_date),
  release_month = month(release_date, label = TRUE, abbr = TRUE),
  release_year = year(release_date)
) |>

# Standardise developer name to title case
mutate(developer = str_to_title(developer)) |>

# Remove columns not needed
select(-name, -release_date, -developer_bio) |>
```

```

# Count developer titles
add_count(developer, name = "dev_titles") |>

# FIX - Convert minutes to hours by dividing by 60
mutate(pt_hrs = round(avg_playtime_mins / 60, 2)) |>

# Create lpeak = log(peak_players), rounded to 2dp
mutate(lpeak = round(log(peak_players), 2)) |>

# Round positive_ratio to integer
mutate(positive_ratio = round(positive_ratio, 0) |> as.integer()) |>

# FIX - Remove arbitrary price cap entirely
mutate(price = round(price, 2)) |>

# FIX - Convert genre_code to factor
mutate(genre_code = factor(genre_code)) |>

# Reorder columns cleanly
select(appid, developer, release_day, release_month, release_year,
       genre, genre_code, price, total_reviews, positive_ratio,
       avg_playtime_mins, peak_players, metacritic_score, platforms,
       pt_hrs, lpeak, dev_titles)

# Check
view(best_practice_clean)
glimpse(best_practice_clean)

```

Rows: 1,441

Columns: 17

```

$ appid          <dbl> 100004, 100005, 100006, 100007, 100009, 100010, 1000~
$ developer      <chr> "Redkite", "Moonlight Games", "Thunderbyte", "Redkit~
$ release_day    <int> 22, 6, 18, 19, 9, 17, 22, 30, 21, 12, 7, 14, 1, 22, ~
$ release_month  <ord> Apr, Nov, Mar, Oct, Nov, Dec, Jun, Dec, Jun, Mar, Ap~
$ release_year   <dbl> 2022, 2020, 2020, 2019, 2023, 2019, 2023, 2018, 2018~
$ genre          <chr> "Action", "Strategy", "Action", "Action", "RPG", "Si~
$ genre_code     <fct> 1, 3, 1, 1, 2, 4, 4, 5, 3, 1, 3, 4, 2, 1, 4, 5, 1, 1~
$ price          <dbl> 56.15, 50.03, 55.21, 62.76, 54.10, 61.38, 51.81, 36.~
$ total_reviews  <dbl> 1699, 562, 812, 966, 293, 435, 747, 948, 461, 465, 4~
$ positive_ratio <int> 96, 82, 76, 93, 61, 85, 60, 56, 97, 99, 56, 83, 91, ~
$ avg_playtime_mins <dbl> 909, 470, 912, 652, 1068, 279, 210, 936, 837, 422, 4~

```

```

$ peak_players      <dbl> 126, 462, 289, 318, 203, 228, 88, 181, 351, 752, 234~
$ metacritic_score  <dbl> 68.0, 71.0, 68.8, 96.7, 85.3, 98.9, 96.8, 65.1, 0.0, ~
$ platforms         <dbl> 3, 1, 2, 1, 1, 1, 3, 3, 2, 3, 2, 1, 2, 1, 2, 2, 2, 3~
$ pt_hrs            <dbl> 15.15, 7.83, 15.20, 10.87, 17.80, 4.65, 3.50, 15.60, ~
$ lpeak             <dbl> 4.84, 6.14, 5.67, 5.76, 5.31, 5.43, 4.48, 5.20, 5.86~
$ dev_titles        <int> 189, 195, 194, 189, 189, 176, 183, 195, 183, 189, 16~

```

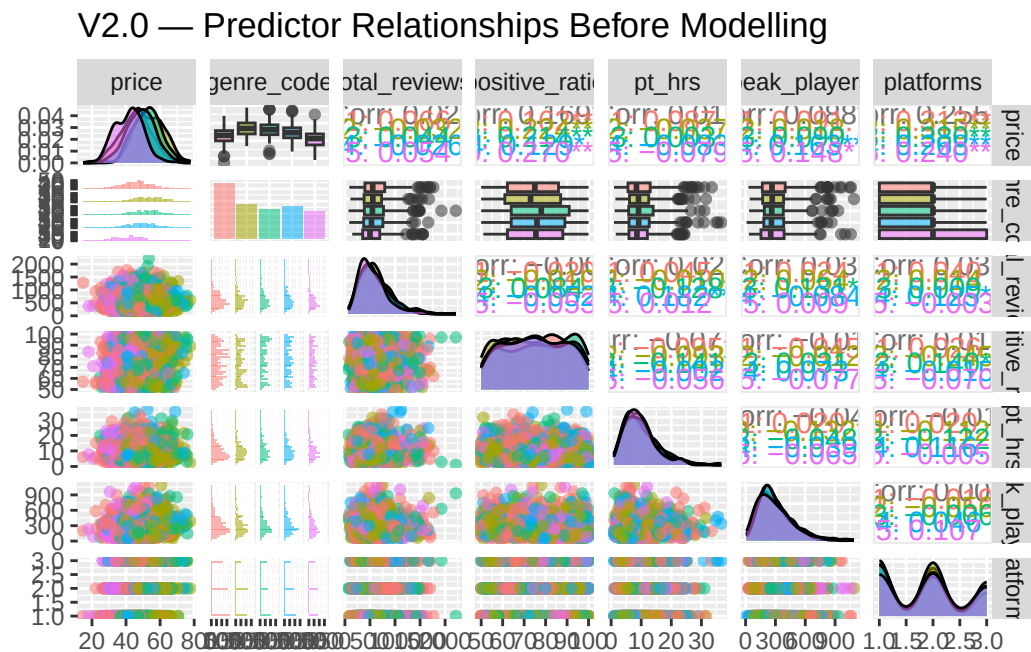
Step 2: Explore Before Modelling

```

library(GGally)

# Explore predictor relationships before fitting the model
best_practice_clean |>
  select(price, genre_code, total_reviews, positive_ratio,
         pt_hrs, peak_players, platforms) |>
  ggpairs(
    title = "V2.0 - Predictor Relationships Before Modelling",
    mapping = aes(alpha = 0.3, colour = genre_code)
  )

```



Step 3: Fit V2 Model and Compare to Legacy

```
# Fit legacy model on student_clean
df <- student_clean
legacy_model <- lm(price ~ genre_code + total_reviews + positive_ratio +
                    pt_hrs + peak_players + platforms, data = df)

# Fit V2 model on best_practice_clean
# genre_code is now a factor
v2_model <- lm(price ~ genre_code + total_reviews + positive_ratio +
               pt_hrs + peak_players + platforms,
               data = best_practice_clean)

# Compare
summary(legacy_model)
```

Call:

```
lm(formula = price ~ genre_code + total_reviews + positive_ratio +
    pt_hrs + peak_players + platforms, data = df)
```

Residuals:

Min	1Q	Median	3Q	Max
-23.3959	0.2298	0.9416	1.5455	3.0379

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	35.6700453	0.8268431	43.140	< 2e-16 ***
genre_code	-0.1870398	0.0773133	-2.419	0.01579 *
total_reviews	0.0002945	0.0004187	0.703	0.48210
positive_ratio	0.0229455	0.0081576	2.813	0.00504 **
pt_hrs	0.0107251	0.0344319	0.311	0.75552
peak_players	0.0014917	0.0006751	2.210	0.02744 *
platforms	0.6662794	0.1488922	4.475	8.84e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.151 on 745 degrees of freedom

Multiple R-squared: 0.04853, Adjusted R-squared: 0.04087

F-statistic: 6.334 on 6 and 745 DF, p-value: 1.637e-06

```
summary(v2_model)
```

Call:

```
lm(formula = price ~ genre_code + total_reviews + positive_ratio +  
    pt_hrs + peak_players + platforms, data = best_practice_clean)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-25.6320	-5.4235	-0.0339	5.4880	25.7947

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	28.9231048	1.4500233	19.947	< 2e-16 ***
genre_code2	8.9973194	0.6125567	14.688	< 2e-16 ***
genre_code3	6.6857016	0.6438131	10.385	< 2e-16 ***
genre_code4	3.4703971	0.6223262	5.576	2.93e-08 ***
genre_code5	-3.9287595	0.6620014	-5.935	3.69e-09 ***
total_reviews	0.0003887	0.0007160	0.543	0.587
positive_ratio	0.1139637	0.0146460	7.781	1.37e-14 ***
pt_hrs	0.0284833	0.0375553	0.758	0.448
peak_players	0.0048972	0.0012280	3.988	7.00e-05 ***
platforms	3.3278884	0.2706047	12.298	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.995 on 1431 degrees of freedom

Multiple R-squared: 0.3099, Adjusted R-squared: 0.3056

F-statistic: 71.41 on 9 and 1431 DF, p-value: < 2.2e-16

Step 4: Polished V2 Plot

```
#Create V2 plot  
best_practice_clean |>  
  ggplot(aes(x = total_reviews, y = price)) +  
  
  geom_point(aes(colour = price), alpha = 0.5, size = 1.2) +  
  
  geom_smooth(method = "lm", se = FALSE, aes(group = 1), linewidth = 0.8) +
```



```

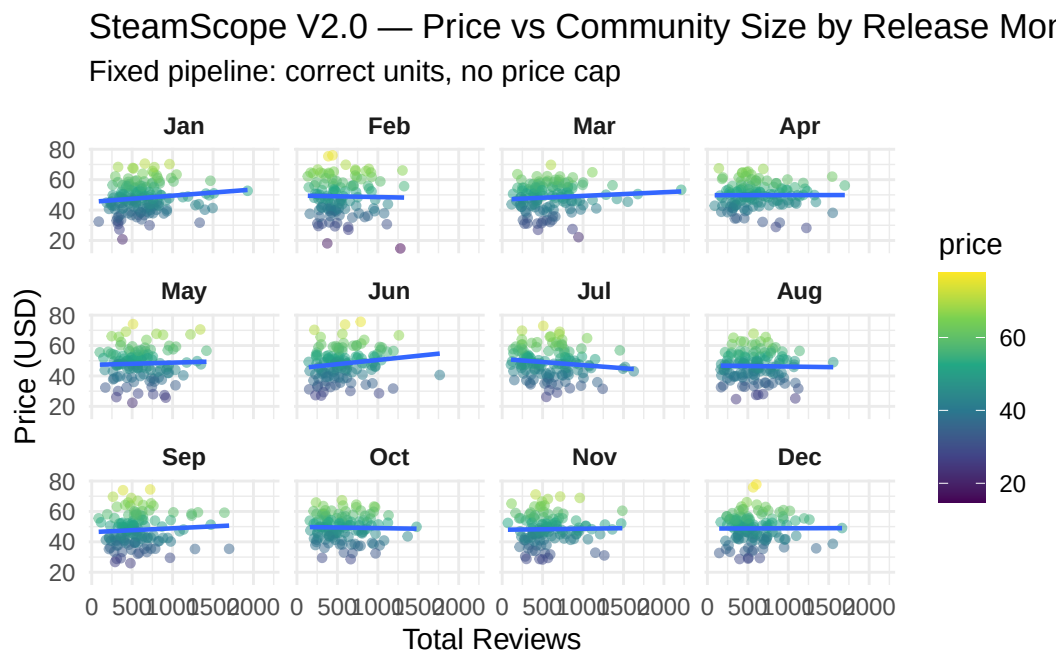
facet_wrap(~ release_month) +

scale_colour_viridis_c() +

labs(
  title    = "SteamScope V2.0 - Price vs Community Size by Release Month",
  subtitle = "Fixed pipeline: correct units, no price cap",
  x        = "Total Reviews",
  y        = "Price (USD)"
) +

theme_minimal() +
theme(strip.text = element_text(face = "bold"))

```



B4: Model Interpretation

Genre 2 has the highest prices, of approximately \$8.99 higher than the base genre, followed by Genre 3 (+\$6.69) and Genre 4 (+\$3.47). Genre 5 has the lowest prices, of about \$3.93 lower than the base genre. This shows clearly different prices depending on genre, where some have been classified as high-end genres while others low-budget ones.

From the regression output above, we saw that the predictors genre, positive_ratio, peak_players, and platforms were significant predictors at $p\text{-value} < 0.05$. Therefore, game pricing is mainly dependent on factors like quality perception (positive reviews), popularity (peak players), distribution channels (platforms) and genre of the game. Whereas, total_reviews and playtime hours are not significant, meaning only engagement volume does not impact pricing decisions for developers.

R^2 coefficient of 0.31 means that the model explains about 31 percent of the game price variation. While the model provides information regarding important factors of the game price, a significant percentage of variation is yet to be explained due to other factors such as brand reputation and marketing strategy.

Conclusion

This analysis successfully reconstructed SteamScope's original pricing pipeline, identified critical data quality issues, and developed an improved Version 2.0 workflow following data analytics best practices. The audit demonstrated that several transformation decisions in the original pipeline reduced model reliability by introducing data distortion, sampling bias, and incorrect variable handling. After correcting these issues, the improved model provided a more reliable understanding of the factors associated with game pricing, increasing explanatory power from $R^2 = 0.049$ to $R^2 = 0.310$.

The findings suggest that pricing decisions are influenced by factors such as game genre, player engagement, review sentiment, and platform availability. However, the model still explains only part of the variation in pricing, indicating that additional factors such as brand strength, marketing strategy, and market positioning should be considered before making final pricing recommendations. Overall, this project highlights the importance of reproducible data pipelines, data quality validation, and statistically sound modelling in producing trustworthy business insights.